

Informe de Trabajo Práctico N° 2

1. Carátula

- Título del Trabajo: Aplicaciones de estructuras jerárquicas y grafos
- Integrantes del equipo:
 - Julián Bautista, Neme Ferrari del Sel
 - Daiana, Dinamarca
- Fecha de entrega: 08/06/2026
- Fecha límite de recuperatorio: 24/06/2026
- Link del repositorio: <https://github.com/JuliFdS/AyED2026c1-Neme-Dinamarca>

2. Introducción

El objetivo de este trabajo práctico es diseñar, implementar, analizar y evaluar estructuras de datos jerárquicas y no lineales para resolver problemas complejos de optimización, flujos de información y gestión masiva de datos en memoria principal.

Para dar solución a las problemáticas planteadas, se aplican conceptos teóricos avanzados de estructuras de prioridad, tales como **montículos binarios** (*heaps*), algoritmos de optimización basados en la **teoría de grafos** (específicamente la búsqueda de Árboles de Expansión Mínima mediante el algoritmo de Prim o Kruskal), y estructuras de búsqueda dinámica auto-balanceadas por altura, implementando un **Árbol Binario de Búsqueda AVL**. A lo largo del desarrollo, se diseñan operaciones críticas de inserción, eliminación, rebalanceo geométrico y recorridos de frontera, evaluando rigurosamente los órdenes de complejidad temporal y espacial mediante la notación formal Big-O.

Este informe presenta el desarrollo, análisis y resolución técnica de la Guía de Trabajos Prácticos N°2, cuyo objetivo central es la aplicación de estructuras jerárquicas y algoritmos de optimización de grafos. El trabajo se encuentra dividido en tres grandes bloques conceptuales: en primer lugar, se modela un sistema de triaje médico genérico aplicando *max-heaps*; en segundo lugar, se construye la interfaz *Temperaturas_DB* abstrayendo la lógica analítica de un árbol auto-balanceado AVL; y en tercer lugar, se resuelve la distribución óptima de información geográfica a través de un árbol de cobertura mínima.

3. Desarrollo de los problemas

3.1. Problema N° 1: Sala de Emergencias

A. Análisis y planteo

El objetivo principal radica en optimizar y priorizar el flujo de atención en una sala de emergencias a través de la modelación e implementación del proceso clínico de triaje. En este escenario crítico, los esquemas de atención tradicionales basados en el orden de llegada (política estricta FIFO) resultan ineficientes e inviables, dado que no discriminan la

urgencia biológica ante riesgos de salud severos. En términos abstractos, las operaciones críticas que determinan la eficiencia del sistema son la inserción dinámica y frecuente de nuevos elementos y la extracción constante del elemento con máxima prioridad (atención del paciente cuyo estado de riesgo es el más urgente).

Para satisfacer estos requerimientos operacionales bajo criterios de alta eficiencia, se seleccionó una **Cola de Prioridad genérica implementada mediante un Montículo Binario Mínimo (Min-Heap)** por composición. Se impone una propiedad de orden estricta: el valor de prioridad de cada nodo padre es siempre menor o igual al de sus nodos hijos. De este modo, se garantiza que el elemento con el valor numérico de riesgo más bajo (donde 1-CRÍTICO representa la máxima urgencia en comparación con el riesgo 3-BAJO) permanezca persistentemente en el nodo raíz para un acceso inmediato.

- Supuestos de la elección: Bajo un flujo constante de datos, un arreglo ordenado requeriría un costo de inserción lineal perjudicial $O(N)$ debido al desplazamiento de memoria física. En contraposición, el Heap binario permite amortizar tanto las inserciones como las bajas en tiempo estrictamente logarítmico, manteniendo el árbol perfectamente balanceado por su propiedad de estructura compacta.
- Segundo criterio (Desempate): Para resolver escenarios de equidad clínica (mismo nivel de riesgo), se introdujo un atributo secuencial indexado (`__orden_llegada`) dentro del objeto, resolviendo el desempate de forma interna y respetando la política FIFO para prioridades equivalentes sin romper la genericidad de la estructura.

B. Implementación / Resolución

La resolución arquitectónica del software se dividió de forma modular con bajo acoplamiento:

1. **Estructura Genérica (heap.py / cola_prioridad):** Define la clase ColaPrioridad que utiliza internamente un Monticulo por composición. La reestructuración geométrica interna utiliza procesos llamados `__flotar` (up-heapify) y `__hundir` (down-heapify). Es completamente abstracta y genérica; delega los criterios de ordenamiento en los operadores relacionales de los objetos que almacena.
2. **Lógica del Dominio (paciente.py):** Implementa la clase Paciente encapsulando los datos y sobrecargando el método mágico `__lt__` (menor que) para comandar al

Heap:

```
def __lt__(self, otro):  
    """  
    Sobrecarga del operador '<' (Less Than).  
    Indica si este paciente tiene más prioridad que 'otro'.  
    """  
  
    # Criterio 1: Riesgo (1 es más prioritario que 2)  
    if self.__riesgo != otro.__riesgo:  
        return self.__riesgo < otro.__riesgo  
  
    # Criterio 2: Si el riesgo es igual, el que llegó antes (menor orden) va primero  
    return self.__orden_llegada < otro.__orden_llegada
```

Fundamentación de Complejidad (Métrica Big-O)

- Inserción: Complejidad $O(\log N)$. Al ingresar un nuevo elemento en un nodo hoja, el algoritmo ejecuta el método `__flotar`. En el peor caso absoluto, recorre la altura máxima del árbol casi completo, la cual está estrictamente acotada a $\log_2 N$.
- Eliminación: Complejidad $O(\log N)$. Extraer el mínimo toma tiempo $O(1)$, pero la remoción física requiere sustituir la raíz por el último elemento y ejecutar el algoritmo `__hundir` para restaurar la propiedad de orden recorriendo las ramas en tiempo logarítmico.

C. Resultados y pruebas

Para verificar la robustez del sistema y dar estricto cumplimiento a las correcciones del cuerpo docente, se ejecutaron verificaciones en dos niveles independientes:

1. Prueba de Uso Aislada del TAD Genérico (Aspecto a evaluar nº 4)

Dando cumplimiento al requerimiento de verificación modular, se incorporó un bloque de prueba autónomo al final del archivo `heap.py` mediante el punto de entrada nativo `if __name__ == "__main__":`. Esta rutina evalúa la estructura de forma aislada de la aplicación utilizando tuplas de datos abstractos. Las pruebas de terminal validaron que la clase `ColaPrioridad` delega y encapsula correctamente las operaciones del `Monticulo` subyacente, insertando elementos, extrayendo los mínimos en orden de prioridad estricto, computando su tamaño con `__len__`, recorriendo en modo lectura con `__iter__` y capturando de forma segura excepciones del tipo `IndexError` ante operaciones inválidas en estructuras vacías.

2. Integración y Simulación Oficial (Aspecto a evaluar nº 2)

Para subsanar la observación sobre el flujo del programa principal, se descartaron los scripts de prueba aleatorios y **se restauró de forma estricta la estructura conductora de la simulación original provista por la cátedra en su repositorio oficial**. La integración de

la nueva estructura se realizó reemplazando de forma transparente la lista secuencial nativa por una instancia de nuestra ColaPrioridad genérica.

Al ejecutar el archivo `main.py` original modificado, el entorno procesó exactamente el mismo lote y flujo cronológico de pacientes diseñado por la universidad. Las trazas de la terminal demostraron el éxito de la sustitución: mientras que el código base atendía por estricto orden de llegada relegando a los enfermos graves, el nuevo sistema interceptó las prioridades dinámicamente. Los pacientes con código `1-CRÍTICO` fueron promovidos de forma inmediata a la raíz de atención, y ante la coincidencia de niveles de riesgo clínico, el sistema aplicó con precisión el desempate por orden de llegada (`__orden_llegada`) procesando los flujos de manera segura, óptima y fiel al diseño original del triaje.

```

24/06/2026 22:51:44

Pacientes que faltan atenderse: 4
    Jorge Belgrano -> 2-moderado (orden de llegada: 6)
    Andrea Perez   -> 3-bajo (orden de llegada: 2)
    Mariela Belgrano      -> 3-bajo (orden de llegada: 5)
    Leandro Rodriguez     -> 3-bajo (orden de llegada: 3)

_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_
_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_

24/06/2026 22:51:45

Pacientes que faltan atenderse: 5
    Leandro Perez   -> 1-crítico (orden de llegada: 7)
    Jorge Belgrano  -> 2-moderado (orden de llegada: 6)
    Mariela Belgrano      -> 3-bajo (orden de llegada: 5)
    Leandro Rodriguez     -> 3-bajo (orden de llegada: 3)
    Andrea Perez   -> 3-bajo (orden de llegada: 2)

_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_
_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_

24/06/2026 22:51:46

*****
Se atiende el paciente: Leandro Perez   -> 1-crítico (orden de llegada: 7)
*****

```

Nota de Análisis: El log evidencia la propiedad del Min-Heap. Al ingresar Leandro Perez con prioridad 1 en el turno 7, la estructura rebalancea de inmediato desplazando de forma

logarítmica el nodo hacia la raíz. En el ciclo de atención subsiguiente, el sistema extrae el mínimo absoluto, despachando al paciente crítico por encima de las urgencias moderadas y bajas que aguardaban previamente en la sala

3.2. Problema N° 2: Temperaturas_DB

A. Análisis y planteo

El problema consiste en modelar e implementar un Tipo Abstracto de Datos (TAD) orientado al almacenamiento y consulta eficiente de un diccionario de datos dinámico indexado por claves cronológicas (fechas). En términos abstractos, la problemática requiere una estructura asociativa que administre un conjunto de pares *clave-valor*, donde la clave posee un orden total estricto (una fecha es biunívocamente anterior, posterior o igual a otra) y el valor contiene la información científica (temperaturas). Las operaciones principales no solo involucran el acceso puntual por clave, sino la extracción secuencial y ordenada de subconjuntos de datos limitados por rangos numéricos acotados.

Las operaciones críticas para este sistema son la **inserción dinámica** (carga de muestras sin duplicados cronológicos, permitiendo la actualización de valores), la **búsqueda exacta** (recuperar la temperatura de un día específico), la **eliminación física** (remoción de registros erróneos) y el **recorrido ordenado acotado** (recuperación secuencial de muestras e intervalos cerrados de fechas para el cálculo de estadísticas de extremos como mínimos y máximos).

Se seleccionó un **Árbol Binario de Búsqueda Auto-Balanceado por Altura (Árbol AVL)**. Esta estructura satisface óptimamente las operaciones críticas debido a que impone una restricción de equilibrio estricta: para todo nodo, la diferencia de altura entre sus subárboles izquierdo y derecho (factor de balance) no puede exceder la unidad ($|BF| \leq 1$).

La elección de un árbol AVL es completamente razonable bajo el supuesto de que el volumen de la entrada de datos científicos es masivo ($N = 120$ muestras iniciales con potencial de escalabilidad a miles de registros) y que el patrón de uso típico del sistema alterna constantemente entre la inserción de nuevas lecturas y consultas de auditoría por rangos. En un árbol binario de búsqueda convencional (BST), una carga de datos pre-ordenada cronológicamente degradaría la estructura a una lista enlazada, llevando el peor caso de búsqueda a un costo lineal ineficiente $O(N)$. El AVL mitiga este riesgo de forma matemática, garantizando que tanto en el caso promedio como en el peor caso absoluto, la altura del árbol permanezca acotada logarítmicamente. Esto asegura tiempos de respuesta predecibles y de alta velocidad para las operaciones de escritura y consulta puntual.

B. Implementación / Resolución

Para cumplir estrictamente con los criterios de calidad de software y modularidad exigidos por la cátedra, la resolución técnica se diseñó bajo una **arquitectura por capas independientes** con bajo acoplamiento:

1. **Capa del Contenedor Abstracto Genérico (ArbolAVL):** Implementa la estructura matemática pura del árbol. Administra objetos `NodoAVL` que constan de una `clave`, un `valor`, referencias a sus hijos (`izq`, `der`) y un atributo numérico de `altura`. Este componente ejecuta los algoritmos recursivos de inserción y eliminación, y aplica transformaciones geométricas conocidas como **Rotaciones Simples** (Izquierda-Izquierda, Derecha-Derecha) y **Rotaciones Dobles** (Izquierda-Derecha, Derecha-Izquierda) para restablecer el equilibrio ante desbalances. No posee conocimiento sobre el contexto del dominio (temperaturas o strings).
2. **Capa de Negocio e Interfaz (Temperaturas_DB):** Actúa como el puente con el usuario. Utiliza una relación de **composición** al instanciar internamente el árbol genérico (`self._arbol = ArbolAVL()`). Su responsabilidad es parsear las cadenas de texto del usuario (`strings`) a objetos comparables `datetime`, realizar validaciones mediante programación defensiva (bloques `try-except`) y calcular los valores extremos sobre los subconjuntos lineales devueltos por el árbol.

Complejidad Temporal (Métrica Big-O)

La altura máxima de nuestra estructura está delimitada matemáticamente por la fórmula:

$$H \leq 1.44 \log_2(N + 2) - 0.328$$

Método de Interfaz	Complejidad Peor Caso	Mecánica Algorítmica y Justificación
<code>guardar_temperatura</code>	$O(\log N)$	Realiza un descenso vertical por ramificación binaria comparando objetos <code>datetime</code> hasta hallar la posición del nodo ($O(\log N)$). Al retornar en la recursión, actualiza alturas y factores de balance, ejecutando un máximo de dos rotaciones lógicas de costo constante $O(1)$.
<code>devolver_temperatura</code>	$O(\log N)$	Emplea búsqueda binaria sobre el árbol balanceado. En cada nodo visitado se descarta la mitad de los elementos restantes. El número máximo de comparaciones equivale a la altura máxima del árbol.

<code>borrar_temperatura</code>	$O(\log N)$	Localizar el nodo toma $O(\log N)$. Ante una eliminación con dos hijos, busca el sucesor inorden (el menor del subárbol derecho), operación acotada por la altura. El proceso recursivo de rebalanceo posterior recorre el camino inverso hacia la raíz en $O(\log N)$.
<code>max_temp_rango / min_temp_rango</code>	$O(N)$	En el peor escenario lógico (un rango que abarque la totalidad de las fechas cargadas), el algoritmo extrae linealmente los elementos mediante un recorrido inorden acotado por condiciones de frontera y evalúa de forma secuencial sus magnitudes térmicas.
<code>devolver_temperaturas</code>	$O(N)$	Ejecuta un recorrido inorden truncado mediante podas lógicas. Si el intervalo abarca todo el universo de datos, visita secuencialmente los N nodos para construir y retornar el listado ordenado cronológicamente.
<code>cantidad_muestras</code>	$O(1)$	Retorna el estado de un atributo contador entero (<code>self.tamanio</code>) sincronizado dinámicamente con las inserciones y remociones exitosas. Evita cualquier recorrido sobre la estructura.

La **complejidad espacial** del sistema es de orden **$O(N)$** en memoria principal, ya que el almacenamiento escala de manera directamente proporcional a la cantidad de elementos físicos N ingresados desde el archivo de texto.

C. Resultados y pruebas

Para validar la correcta implementación del TAD y el comportamiento de la base de datos ante la carga masiva y consultas concurrentes, se diseñó un lote de prueba interactivo en el módulo `main.py`. El sistema operativo localizó y procesó con éxito el archivo físico de mediciones de Kevin Kelvin proporcionado por la cátedra.

Datos de la Corrida de Prueba en Terminal:

- **Carga Masiva Inicial:** El script parseó y cargó de manera exitosa un volumen de **120 muestras térmicas** en la estructura AVL.
- **Búsqueda Puntual:** Al realizar la consulta exacta para la fecha **"26/01/2025"**, el sistema interceptó el nodo correspondiente de forma logarítmica y devolvió la temperatura correcta de **35.8 °C**.
- **Inserción y Actualización Dinámica:** Se insertó un registro en una fecha nueva, verificándose que el atributo **tamaño** incrementó de manera controlada a **121 muestras**. Seguidamente, se probó la sobreescritura de un valor preexistente, confirmándose que la estructura impidió la duplicación de claves cronológicas y actualizó el dato térmico de forma segura.
- **Consultas por Rangos e Inorden:** Se solicitó un listado ordenado en el intervalo cerrado entre el **"13/01/2025"** y el **"05/03/2025"**. El recorrido inorden del árbol AVL recuperó secuencialmente los elementos, ordenándolos de manera cronológica perfecta (pasando correctamente del 31 de enero al 1 de febrero). Sobre este subconjunto, las funciones estadísticas reportaron con precisión una temperatura máxima de **39.9 °C** y una mínima de **11.9 °C**.
- **Eliminación y Rebalanceo:** Se ejecutó la remoción física de una muestra. El sistema reestructuró internamente los subárboles manteniendo la condición de equilibrio AVL, decrementó la cantidad de muestras de regreso a **120** y, al intentar consultar nuevamente la fecha dada de baja, respondió de forma segura devolviendo un valor nulo (**None**), constatando la inexistencia del registro.

```
PS C:\FIUNER\2026\AyED\AyED2026C1-Neme-Dinamarca\TrabajoPractico_2\proyecto2> & C:\Users\daini\AppData\Local\Programs\Python\Python314\python.exe c:/FIUNER/2026/AyED/AyED2026C1-Neme-Dinamarca/TrabajoPractico_2/proyecto2/main.py
● Éxito: Se cargaron los datos desde data\muestras.txt

=====
TESTEO DE INTERFAZ - TEMPERATURAS DB
=====
1. Cantidad de muestras iniciales: 120
2. Buscando temperatura para 26/01/2025: 35.8 °C
3. Guardando/Actualizando temperaturas...
   Nueva cantidad de muestras: 121
   Temperatura actualizada de 26/01/2025: 35.8 °C
4. Consultas en rango [13/01/2025 inclusive hasta 05/03/2025 inclusive]:
   - Temp Máxima en rango: 39.9 °C
   - Temp Mínima en rango: 11.9 °C
   - Extremos (Min, Máx) en rango: (11.9, 39.9)
5. Listado de muestras en rango [13/01/2025 - 05/03/2025]:
13/01/2025: 33.6 °C
14/01/2025: 34.6 °C
15/01/2025: 36.9 °C
16/01/2025: 27.5 °C
17/01/2025: 15.3 °C
18/01/2025: 32.8 °C
19/01/2025: 14.4 °C
20/01/2025: 19.4 °C
21/01/2025: 38.4 °C
22/01/2025: 37.5 °C
23/01/2025: 34.3 °C
24/01/2025: 29.7 °C
25/01/2025: 36.3 °C
26/01/2025: 35.8 °C
27/01/2025: 22.2 °C
```

```
28/01/2025: 11.9 °C
29/01/2025: 18.9 °C
30/01/2025: 39.0 °C
31/01/2025: 33.2 °C
01/02/2025: 28.8 °C
02/02/2025: 26.9 °C
03/02/2025: 30.6 °C
04/02/2025: 37.8 °C
05/02/2025: 14.8 °C
06/02/2025: 35.6 °C
07/02/2025: 20.8 °C
08/02/2025: 31.8 °C
09/02/2025: 22.8 °C
10/02/2025: 39.9 °C
11/02/2025: 25.4 °C
12/02/2025: 38.9 °C
13/02/2025: 33.4 °C
14/02/2025: 16.9 °C
15/02/2025: 27.8 °C
16/02/2025: 35.4 °C
17/02/2025: 31.3 °C
18/02/2025: 15.8 °C
19/02/2025: 30.3 °C
20/02/2025: 36.6 °C
21/02/2025: 26.3 °C
22/02/2025: 22.5 °C
23/02/2025: 19.8 °C
24/02/2025: 29.8 °C
25/02/2025: 39.7 °C
26/02/2025: 23.4 °C
27/02/2025: 38.1 °C
28/02/2025: 28.3 °C
01/03/2025: 21.8 °C
02/03/2025: 37.0 °C
03/03/2025: 32.4 °C

04/03/2025: 27.3 °C
05/03/2025: 24.3 °C

6. Borrando temperatura del 20/05/2026...
Cantidad de muestras final: 120
Buscando la fecha borrada: none
=====
```

3.3. Problema N° 3: Palomas Mensajeras

A. Análisis y planteo

El problema requiere resolver de manera óptima la distribución y replicación de información geoespacial desde un nodo central hacia un conjunto cerrado de comunidades periféricas, minimizando de forma absoluta el consumo de recursos logísticos (distancia total recorrida en leguas) y asegurando que cada nodo sea alertado una única vez (evitando redundancias o la recepción de mensajes duplicados).

En términos abstractos, este escenario se modela mediante la abstracción de la **Teoría de Grafos** como un **Grafo Ponderado (Pesado), No Dirigido y Conexo** $G = (V, E, W)$. Los vértices o nodos (V) representan las 22 aldeas del dominio, las aristas (E) constituyen los canales de vuelo viables para las palomas mensajeras, y las ponderaciones (W) indican el costo lineal en leguas entre cada par de comunidades. Bajo este modelo, el objetivo abstracto consiste en hallar un **Árbol de Expansión Mínima** (MST - *Minimum Spanning Tree*), es decir, un subgrafo acíclico que conecte a la totalidad de los vértices minimizando la sumatoria matemática de sus pesos.

Las operaciones críticas identificadas son el **recorrido topológico de la red** y la **selección iterativa y codiciosa (*greedy*) de la arista de menor peso absoluto** que conecte al conjunto de nodos ya explorados con un vértice externamente aislado.

Para satisfacer estas necesidades operacionales, se seleccionó el **Algoritmo de Prim** (o Prim-Jarník). Esta elección es razonable bajo el supuesto de que el problema define un punto de origen fijo, central y predeterminado denominado "*Peligros*". A diferencia de otros esquemas de optimización global (como el algoritmo de Kruskal), el algoritmo de Prim expande la cobertura de manera incremental y radial a partir de un nodo raíz inicial, lo que

se acopla perfectamente a la naturaleza física de la difusión de la noticia comandada por William.

B. Implementación / Resolución

La resolución del software se diseñó de forma altamente modular y con un bajo acoplamiento, explotando las ventajas de la reutilización de código a través de las siguientes capas técnicas:

1. **TAD Grafo ([grafo.py](#)):** Modela la topología de la red mediante una estructura de **Lista de Adyacencia** implementada sobre diccionarios internos de Python indexados por los nombres (strings) de las aldeas. Esto garantiza un costo de almacenamiento lineal $O(V+E)$ y accesos de tiempo constante $O(1)$ para recuperar los vecinos inmediatos de cualquier comunidad.
2. **Algoritmo de Prim y Abstracción de Prioridad ([logica_palomas.py](#)):** Para evitar costosas búsquedas lineales $O(V)$ al explorar las fronteras del árbol, el algoritmo se optimizó **co-asociando por composición el TAD ColaPrioridad (Min-Heap) desarrollado genéricamente en el Problema N° 1**. La estructura almacena tuplas ordenadas bajo el formato numérico ([peso](#), [nodo_origen](#), [nodo_destino](#)). En cada iteración, el algoritmo extrae la arista mínima del *Heap*, marca el nodo destino como visitado para neutralizar ciclos y actualiza los punteros de procedencia en una estructura de precedencias.

C. Resultados y pruebas

Para validar la efectividad de la solución, se alimentó al programa con el archivo físico real [aldeas.txt](#) provisto por la cátedra. El sistema implementó un manejo defensivo de cadenas para filtrar y omitir líneas inválidas o vacías, construyendo exitosamente el grafo ponderado en memoria.

Al ejecutar la optimización central, el algoritmo de Prim procesó de forma segura la red desde el nodo raíz "*Peligros*", hallando la topología de distribución óptima. Los resultados formateados en la consola arrojaron una ordenación alfabética perfecta de las comunidades y determinaron con absoluta precisión matemática la procedencia de la transmisión (de qué vecina recibe la noticia) y la ramificación de réplicas autorizadas (a qué vecinas envía copias):

Trazas de la Distribución Final y Métrica de Recursos (Terminal):

```
=====
SISTEMA DE COMUNICACIÓN: PALOMAS WILLIAM
=====
Aldeas registradas (22):
Aceituna, Buenas Noches, Cebolla, Consuegra, Diosleguarde, El Cerrillo, Elciego, Espera, Hortijos, Humilladero, La Aparecida, La Pera, Lomaseca, Los Infiernos, Malcocinado, Melón, Pancrudo, Peligros, Pepino, Silla, Torralta, Villaviciosa
-----
ALDEA          | RECIBE NOTICIA DE      | ENVÍA RÉPLICAS A
-----
Aceituna        | Malcocinado            | ---
Buenas Noches   | La Aparecida           | Cebolla
Cebolla         | Buenas Noches          | Pancrudo
Consuegra       | Malcocinado            | ---
Diosleguarde    | Malcocinado            | Elciego
El Cerrillo     | Lomaseca               | Malcocinado
Elciego         | Diosleguarde           | Melón
Espera         | La Pera               | ---
Hortijos        | Humilladero            | ---
Humilladero     | Torralta               | Hortijos
La Aparecida    | Peligros               | Buenas Noches, Silla
La Pera        | Los Infiernos          | Espera
Lomaseca       | Peligros               | Los Infiernos, Pepino, El Cerrillo
Los Infiernos  | Lomaseca               | La Pera
Malcocinado    | El Cerrillo            | Consuegra, Aceituna, Diosleguarde
Melón          | Elciego                | ---
Pancrudo       | Cebolla                | ---
Peligros       | (Punto de Origen)      | La Aparecida, Lomaseca
Pepino         | Lomaseca               | ---
Silla          | La Aparecida           | Torralta
Torralta       | Silla                  | Villaviciosa, Humilladero
Villaviciosa   | Torralta               | ---
-----
SUMA TOTAL DE DISTANCIAS RECORRIDAS: 94 leguas.
=====
PS C:\FIUNER\2026\AyED\AyED2026\1-Neme-Dinamarca\TrabajoPractico_2\proyecto3>
```

La ejecución concluyó de forma exitosa sin fugas de datos ni bucles infinitos. Las pruebas empíricas constataron que **el costo mínimo absoluto para alertar a las 21 comunidades desde la base central es de exactamente 94 leguas**, garantizando a la agencia "Palomas William" el máximo ahorro de recursos logísticos mediante una red de comunicación óptima, acíclica y completamente estable.

4. Análisis de resultados específicos / Discusión

En este apartado se consolida el estudio analítico y matemático formal de los tres problemas desarrollados en la guía. A través de la métrica Big-O, se evalúa el rendimiento temporal y espacial de las estructuras de datos jerárquicas y no lineales implementadas, discutiendo el impacto directo de la modularidad y las restricciones geométricas sobre la eficiencia en memoria principal.

A. Análisis de Complejidad Temporal del Peor y Caso Promedio (Métrica Big-O)

Para garantizar la estabilidad del software ante volúmenes masivos de datos, se exponen a continuación las tablas de complejidad de los Tipos Abstractos de Datos (TAD) diseñados y reutilizados a lo largo de este trabajo práctico.

1. Tabla de Complejidad: Estructura de la Sala de Emergencias (Min-Heap Genérico - Problema 1)

Operación / Método	Complejidad Peor Caso	Complejidad Caso Promedio	Justificación Técnica y Mecánica Estructural
<code>insertar(item)</code>	$O(\log N)$	$O(1)$ amortizado	El elemento se añade al final del arreglo base ($O(1)$). En el peor caso (paciente con prioridad máxima absoluta), debe ascender verticalmente mediante el proceso <code>flotar</code> . Como el montículo es un árbol casi completo, la altura máxima está estrictamente acotada por $\log_2 N$. En el caso promedio, la densidad de nodos en los niveles inferiores reduce estadísticamente los intercambios necesarios a un costo constante.

<code>extraer_minimo()</code>	$O(\log N)$	$O(\log N)$	La raíz se descarta en tiempo constante y se reemplaza por el último nodo hoja. Para restaurar la propiedad de orden, este nodo transitorio debe filtrarse de manera descendente mediante el proceso <code>hundir</code> , realizando divisiones binarias sucesivas a lo largo de los niveles del árbol, lo que demanda un costo proporcional a su altura tanto en el peor como en el caso promedio.
<code>esta_vacia()</code>	$O(1)$	$O(1)$	Operación de acceso directo. Interroga la referencia vacía o el estado de longitud de la lista interna sin inspeccionar los elementos individuales.
<code>__len__()</code>	$O(1)$	$O(1)$	Retorna el valor de un atributo entero privado mapeado que se incrementa o decrementa de forma sincrónica con cada operación, evitando recorridos lineales.

2. Tabla de Complejidad: Diccionario de Datos Climáticos (Árbol AVL - Problema 2)

La restricción de equilibrio estricta del Árbol AVL garantiza de forma matemática que su altura máxima se encuentra acotada por la relación: $H \leq 1.44 \log_2(N + 2) - 0.328$, impidiendo la degradación lineal de la estructura (Ver tabla en Problema 2)

B. Análisis de Complejidad en la Optimización de Grafos (Problema 3)

El análisis del **Algoritmo de Prim** para el tendido de la red de palomas mensajeras demuestra de forma contundente los beneficios de la reutilización modular de componentes compartidos.

Si el algoritmo se diseñara utilizando un enfoque tradicional basado en arreglos o listas lineales para explorar la frontera del subgrafo, la selección de la arista con el peso mínimo requeriría una inspección secuencial completa de los vértices no visitados, arrojando una complejidad temporal de $O(V^2)$.

En la solución técnica implementada, se integró por composición la estructura ColaPrioridad (Min-Heap) desarrollada originalmente para la sala de emergencias. Esta simbiosis de componentes modifica radicalmente la eficiencia matemática:

- La extracción de la arista de costo mínimo en leguas toma tiempo logarítmico $O(\log V)$ una vez por cada vértice incorporado.
- El análisis e inserción en el Heap de las aristas incidentes hacia las aldeas vecinas inexploradas toma un costo acotado por $O(\log V)$ para cada arista procesada.

Como resultado directo, la complejidad temporal en el peor caso absoluto queda acotada de forma estricta por el orden:

Complejidad Temporal (Prim con Min-Heap) = $O(E \log V)$

Donde E representa la cantidad de rutas viables (aristas) registradas en el archivo físico y V el número total de comunidades (22). Dado que la red geográfica de aldeas constituye un grafo disperso (donde el número de aristas E está muy alejado del límite cuadrático V^2), esta optimización mitiga el crecimiento polinomial del tiempo de cómputo, validando la eficiencia y la arquitectura de software adoptada.

C. Análisis de Complejidad Espacial General

La complejidad espacial del proyecto se evaluó de forma transversal en memoria principal:

- **Problema 1 y 2 (Montículo y AVL):** Presentan una complejidad espacial de orden $O(N)$, escalando de forma estrictamente lineal y proporcional a la cantidad de pacientes activos en sala y al volumen de muestras climáticas indexadas (120 registros iniciales), respectivamente.
- **Problema 3 (Grafos):** Al estructurar la red mediante una **Lista de Adyacencia** basada en diccionarios anidados, el espacio en memoria está acotado por $O(V + E)$, optimizando los recursos lógicos al almacenar únicamente las conexiones reales existentes entre las aldeas y descartando los espacios vacíos propios de una matriz de adyacencia.

D. Discusión General y Hallazgos Algorítmicos

La experimentación con estructuras de datos no lineales y jerárquicas permite extraer las siguientes conclusiones críticas:

1. **Control Defensivo de la Degradación:** Las estructuras lineales sufren severos cuellos de botella temporales ($O(N)$) ante inserciones o búsquedas masivas dinámicas. Las estructuras jerárquicas actúan como mecanismos geométricos de control: el árbol AVL, mediante sus rotaciones lógicas, impide activamente que la carga cronológica convierta el árbol en una lista enlazada; el montículo binario, por su propiedad de completitud por niveles, autogarantiza que su altura nunca escape de los límites logarítmicos.

2. **Cohesión y Reutilización Modular:** El éxito en el rendimiento del Problema 3 (Palomas Mensajeras) demuestra que la abstracción matemática de un TAD genérico (ColaPrioridad) puede resolver problemáticas de dominios completamente inconexos (Triage médico y logística de comunicaciones) sin alterar una sola línea de su código estructural. La separación de responsabilidades por capas e interfaces limpias se consolida como una práctica indispensable en el diseño de algoritmos eficientes y software de alta calidad.

5. Declaración de Uso Ético de Inteligencia Artificial

Herramientas utilizadas:

- **Gemini** (Google).
- **Claude** (Anthropic)

Descripción del uso:

- **Generación de ideas/estructura:** Se consultaron los modelos para debatir y modelar el segundo criterio de selección secuencial (`__orden_llegada`) en el triaje médico y para diagramar de forma lógica la reutilización por composición de la Cola de Prioridad genérica dentro del algoritmo de grafos.
- **Asistencia en código/redacción:** Se emplearon las herramientas para estructurar la lógica matemática de flotación y hundimiento del *Min-Heap*, depurar errores de importación en el entorno de ejecución, redactar formalmente las especificaciones de pre y postcondiciones de las interfaces, y pulir la redacción académica general del informe técnico. Para la instancia del recuperatorio, se incorporó el uso de Claude con el fin de optimizar el análisis y la resolución quirúrgica de los *issues* y correcciones marcados por el cuerpo docente.
- **Consulta teórica:** Se validaron los fundamentos teóricos del Algoritmo de Prim en grafos dispersos, las propiedades de rebalanceo geométrico del Árbol AVL y las cotas de complejidad de la estructura de Montículos Binarios.

Declaración de responsabilidad:

Los integrantes del grupo declaramos que hemos revisado y validado personalmente toda la información y código sugerido por las herramientas de IA. Entendemos los conceptos presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final es producto de nuestro propio análisis y criterio.

6. Trabajo en equipo

A. Planificación y Estimación Inicial del Tiempo

Antes de comenzar con el desarrollo del proyecto, el equipo realizó una proyección del esfuerzo requerido para resolver los tres problemas y confeccionar la documentación

técnica. Se estimó una dedicación total de **24 horas de trabajo**, distribuidas a lo largo de las dos semanas de plazo de la siguiente manera:

- **Investigación y Diseño Teórico (4 horas):** Lectura de consignas, análisis del código base del simulador proporcionado por la cátedra y repaso de las operaciones de balanceo AVL.
- **Desarrollo de Software y Codificación (12 horas):** Escritura de los TADs genéricos, modularización por capas e implementación de algoritmos de optimización.
- **Pruebas y Depuración (4 horas):** Validación de casos de prueba, ejecución de archivos de control y resolución de errores de entorno.
- **Redacción del Informe Técnico (4 horas):** Confección de tablas Big-O, especificación de pre/postcondiciones y documentación de resultados.

B. Organización del Trabajo, Roles y Puesta en Común

A. Estimación Inicial de Tiempos y Planificación

Al inicio del proyecto, el equipo realizó una proyección de carga horaria estimando un total de **26 horas de trabajo conjunto**, distribuidas a lo largo de dos semanas con el siguiente cronograma planificado:

- **Investigación y Lectura de Consignas:** 4 horas (2 horas por integrante) para la comprensión del repositorio base y la recopilación de fundamentos teóricos.
- **Desarrollo y Codificación:** 12 horas (6 horas por integrante) para la programación de las estructuras AVL, Montículo y el Algoritmo de Prim.
- **Depuración, Validación y Pruebas:** 4 horas (2 horas por integrante) para corroborar la salida de los simuladores.
- **Redacción del Informe Técnico:** 6 horas (3 horas por integrante) trabajando sobre una plantilla compartida.

B. Organización del Trabajo, Roles y Canales de Comunicación

El proceso de desarrollo comenzó de forma presencial e interactiva en el aula de informática el día de la presentación oficial del trabajo práctico por parte de la cátedra. En dicha instancia, se analizaron los archivos iniciales y se determinó una división de responsabilidades basada en la modularidad del proyecto:

- **Daiana Dinamarca:** Asumió la responsabilidad del **Problema N° 2 (Base de Datos Climática)**. Se encargó del diseño, codificación y testeo del contenedor genérico **ArbolAVL** y de la capa de interfaz **TemperaturasDB**, gestionando la lógica de balanceo y el procesamiento del archivo físico de muestras.
- **Julián Neme:** Asumió la responsabilidad del **Problema N° 1 (Sala de Emergencias)** y del **Problema N° 3 (Palomas Mensajeras)**. Se encargó de codificar la lógica del montículo base por composición, sobrecargar los operadores

de la clase **Paciente** e implementar el Algoritmo de Prim vinculando la lista de adyacencias con la cola de prioridad.

Mecanismos de Coordinación: Debido a la naturaleza no presencial del desarrollo subsiguiente, el trabajo se llevó a cabo de manera remota. El canal de comunicación principal y diario fue la aplicación de mensajería instantánea **WhatsApp**, utilizada para coordinar el estado de los módulos, compartir capturas de errores de la consola y unificar criterios de diseño. La puesta en común y consolidación del código se realizó mediante confirmaciones (*commits*) en el repositorio controlado por Git, mientras que la redacción del informe técnico se ejecutó en tiempo real de forma colaborativa mediante un **documento compartido en Google Docs**.

C. Proceso de Revisión y Coordinación en Instancia de Recuperatorio

Ante las correcciones emitidas por el cuerpo docente en la primera entrega, el equipo activó un protocolo de revisión exhaustiva enfocado en el control de calidad del software y la reestructuración arquitectónica:

- **Daiana Dinamarca** asumió el rol de revisora general del proyecto, encargándose de auditar la totalidad del código fuente de los tres problemas, reescribir los encabezados de importación en **heap.py** para solucionar fallos del entorno de ejecución y verificar el estricto cumplimiento de las pre y postcondiciones en todas las interfaces de los TAD.
- **Julián Neme** trabajó en comunicación continua y sincrónica vía WhatsApp para validar los cambios aplicados en la simulación de triaje médico, asegurando la restauración fiel del archivo conductor **main.py** original provisto por la universidad.
- **El Informe Técnico Final** y la resolución de la Sección 4 y 5 fueron redactados y pulidos de forma conjunta por ambos integrantes mediante la plataforma web compartida, discutiendo y unificando los criterios analíticos de la métrica Big-O.

D. Comparación de Tiempos: Estimado vs. Real

El tiempo efectivamente dedicado al trabajo práctico ascendió a un total aproximado de **41 horas combinadas**, evidenciando un desvío significativo de +15 horas respecto a la planificación inicial.

Fase de Trabajo	Tiempo Estimado	Tiempo Real (Involucrando Recuperatorio)
Investigación y Lectura	4 horas	5 horas
Desarrollo y Codificación	12 horas	16 horas

Validación y Pruebas	4 horas	11 horas
Redacción e Informe	6 horas	9 horas
TOTAL	26 horas	41 horas

Análisis de Causas de los Desvíos Técnicos:

1. **Dificultades Técnicas e Imports Cruzados (+4 horas en Desarrollo):** La resolución del encapsulamiento de la `ColaPrioridad` por sobre el `Monticulo` y la gestión de rutas relativas con el punto `(.)` generaron fallos de tipo `ModuleNotFoundError` en la consola que requirieron mayor tiempo de investigación y el diseño defensivo de un bloque `try-except` dinámico.
2. **Auditoría y Pruebas del Recuperatorio (+7 horas en Validación):** El requerimiento del Aspecto a evaluar N° 4 obligó al grupo a diseñar de forma aislada pruebas unitarias bajo bloques `if __name__ == "__main__":` en cada módulo, lo que incrementó significativamente las horas destinadas a la consola y a la simulación en tiempo real.
3. **Alineación del Informe (+3 horas en Redacción):** La necesidad de unificar de forma matemática formal todo el análisis Big-O de los tres problemas dentro de un único apartado estructural (Sección 4) demandó sesiones adicionales de redacción colaborativa para cumplir estrictamente con los criterios de rigurosidad científica de la cátedra.

7. Conclusiones

Al concluir el desarrollo y la revisión de este trabajo práctico, se constata el cumplimiento satisfactorio y riguroso de la totalidad de los objetivos académicos e informáticos trazados por la cátedra. La resolución de las problemáticas planteadas permitió trasladar los modelos matemáticos abstractos a soluciones de software concretas y altamente eficientes en memoria principal, evidenciando cómo la selección estratégica de estructuras de datos no lineales transforma radicalmente el rendimiento de un sistema.

A lo largo de la guía, se experimentaron y superaron diversas dificultades técnicas de alta complejidad:

- En los **Problemas 1 y 3**, la principal barrera radicó en el manejo preciso de los índices y referencias posicionales dentro del arreglo dinámico del Montículo Binario durante los procesos de reestructuración geométrica (`__flotar` y `__hundir`), lo cual se subsanó mediante sesiones exhaustivas de depuración y el diseño de pruebas aisladas paso a paso.
- En el **Problema 2**, el desafío central consistió en la correcta implementación de los algoritmos recursivos para las rotaciones simples y dobles del Árbol AVL,

asegurando la actualización sincrónica de las alturas de los nodos sin corromper la propiedad de orden binario.

- En la **Fase del Recuperatorio**, la mayor dificultad fue de carácter arquitectónico: desacoplar las estructuras genéricas de las reglas de negocio del dominio para cumplir con el estándar de modularidad por capas y resolver la compatibilidad del entorno de ejecución frente a las importaciones relativas de los paquetes. Este obstáculo se superó mediante un diseño defensivo basado en bloques de control de excepciones (`try-except`) dinámicos.

El aprendizaje técnico y profesional más significativo consolidado en este proyecto es la necesidad imperiosa de concebir el software mediante abstracciones genéricas y contratos lógicos estrictos (pre y postcondiciones). Comprender que una misma estructura abstracta (`ColaPrioridad`) puede regular con la misma eficiencia un flujo de triaje clínico asíncrono y un algoritmo de optimización topológica de grafos dispersos (*Prim*), sin alterar una sola línea de su código base, demuestra el valor de la reutilización modular y el encapsulamiento.

Finalmente, el proceso reafirmó que la rigurosidad en la validación experimental autónoma y la comunicación técnica continua dentro de un equipo son pilares indispensables para garantizar la calidad de software y el éxito en la ingeniería de algoritmos.

8. Referencias y Bibliografía

A. Libros y Material de Lectura Principal

- **Miller, B. N., & Ranum, D. L.** *Solución de Problemas con Algoritmos y Estructuras de Datos usando Python*. Runestone Academy. Disponible en: <https://runestone.academy/ns/books/published/pythoned/index.html>
 - *Nota de uso:* Consultado formalmente para la fundamentación teórica de Montículos Binarios (Heaps), árboles auto-balanceados AVL (factores de equilibrio y mecánicas de rotación), y la implementación optimizada del Algoritmo de Prim mediante colas de prioridad.

B. Apuntes y Documentación Interna de la Cátedra

- **Cátedra de Algoritmos y Estructuras de Datos (AyED).** *Apuntes Teóricos Completo - Estructuras de Datos No Lineales, Jerárquicas y Grafos*. Facultad de Ingeniería, Universidad Nacional de Entre Ríos (FIUNER), 2026. Documento de trabajo interno. Disponible en: <https://docs.google.com/document/d/1pO310W2b2Qj0Kq8uzstduWqkdRepy2GH4QS08i1aXps/edit>
 - *Nota de uso:* Utilizado como marco regulatorio primordial para las restricciones de diseño arquitectónico, el planteo estricto de

pre/postcondiciones y las pautas institucionales de evaluación del Trabajo Práctico N° 2.

C. Repositorios Oficiales y Fuentes de Datos

- **FIUNER-LICA.** *Repositorio de Códigos Iniciales de Soporte para Estudiantes - Trabajo Práctico N° 2.* Laboratorio de Informática y Ciencias Aliadas, Universidad Nacional de Entre Ríos. Disponible en el repositorio de GitHub:
<https://github.com/FIUNER-LICA/ayed-tps-codigos-iniciales-para-estudiantes>
 - *Nota de uso:* Fuente del script conductor de simulación original (`main.py` de la Sala de Emergencias) restaurado para la aprobación del Aspecto N° 2, y de los archivos de datos físicos masivos (`muestras.txt` y `aldeas.txt`) empleados para las pruebas empíricas de los Problemas 2 y 3.

D. Documentación Técnica de Lenguajes

- **Python Software Foundation.** *The Python Standard Library - `datetime` (Basic date and time types) & Built-in Types (`__lt__` operator overloading).* Python 3.14 Documentation, 2026. Disponible en: <https://docs.python.org/3/>
 - *Nota de uso:* Revisado para asegurar la correcta manipulación cronológica de las claves en el Árbol AVL y la sobrecarga nativa de operadores relacionales en el triaje médico.